להלן מספר קטעי main וקוד עזר (כל main עומד בפני עצמו).

```
// M A I N - 1
int main() {
  MyList<int> m1;
  for(int i = 1; i <= 2; ++i) m1.push_front(i);
  m1.push_front(3);
  for(int i = 1; i <= 2; ++i) m1.push_back(i);
  // m1.apply( [](auto& val){ val *= 2;} );
  m1.apply( [](const auto& val){cout << val << endl;} );
}
```

```
// M A I N – 2 (+ 2 additional structs)
struct A {
  friend ostream& operator<<(ostream& out, const A& a) {
    return out << "A";
  }
  virtual ~A() { cout << "~A" << endl; }
};

struct B : public A {
  friend ostream& operator<<(ostream& out, const B& b) {
    return out << "B";
  }
  virtual ~B() { cout << "~B" << endl; }
};

int main() {
  MyList<B> m2;
  m2.push_back(A());
  m2.push_back(B());
  m2.apply( [](const auto& val){ cout << val << endl;} );
}
```

```
// M A I N - 3
int main() {
  MyList<std::string> m3;
  m3.push_back("Hello");
  m3.push_back("World");
  MyList<MyList<std::string>> m4;
  // m4.push_front(m3);
  // m4.push_back(m3);
  m4.apply( [](const auto& list){
    list.apply( [](const auto& val){ cout << val << endl; }); } );
}
```

```
 1. template<class E> class MyList {
 2. class Node {
 3.     friend class MyList<E>;
 4.     E data;
 5.     Node* next = nullptr;
 6.   }; // end of class Node
 7.   Node* head = new Node;
 8.   Node* tail = head;
 9.   MyList(const MyList&) = default;
10. public:
11.   MyList() = default;
12.   MyList& operator=(const MyList&) = delete;
13.   void push_front(const E& data) {
14.     Node* node = new Node;
15.     node->data = data;
16.     node->next = head->next;
17.     head->next = node;
18.     if(head->next == nullptr) tail = node;
19.   }
20.   void push_back(const E& data) {
21.     if(head->next == nullptr) {
22.         push_front(data); return;
23.     }
24.     MyList temp(*this);
25.     temp.head = temp.head->next;
26.     temp.push_back(data);
27.     temp.head = nullptr;
28.   }
29.   ~MyList() {
30.     Node *node = head, *next;
31.     while(node != nullptr) {
32.         next = node->next;
33.         delete node;
34.         node = next;
35.     }
36.   }
37.   template<class Function>
38.   void apply (Function f) const {
39.     Node* node = head->next;
40.     while(node != nullptr) {
41.         f(node->data);
42.         node = node->next;
43.     }
44.   }
45. };
```

<u>נתון</u>: הקוד תקין וניתן לכתוב עבורו main תקין שיעשה שימוש בכל הפונקציות שהוגדרו.
בעמוד הבא מופיעים קטעי main רלבנטיים.

קטע קוד B – לבחינה פרונטלית בלבד

```
 1. template<class E> class MyList {
 2.   class Node {
 3.     friend class MyList<E>;
 4.     E data;
 5.     Node* next = nullptr;
 6.     ~Node() {delete next;}
 7.   }; // end of class Node
 8.   Node* head = new Node;
 9. public:
10.   MyList() = default;
11.   MyList(const MyList&) = delete;
12.   void add(const E& data) {
13.     Node* node = new Node;
14.     node->data = data;
15.     node->next = head->next;
16.     head->next = node;
17.   }
18.   ~MyList() { delete head; }
19.   class iterator {
20.       friend class MyList;
21.       Node* node;
22.     public:
23.       E& operator*() {
24.           return node->data;
25.       }
26.       bool operator!=(iterator other) const {
27.           return other.node != node;
28.       }
29.       iterator operator++() {
30.           node = node->next;
31.           return *this;
32.       }
33.   };
34.   iterator begin() {
35.       iterator i;
36.       i.node = head->next;
37.       return i;
38.   }
39.   iterator end() {
40.       iterator i;
41.       i.node = nullptr;
42.       return i;
43.   }
44. };
        // continues next page
```

```
45. class A {
46. public:
47.   virtual ~A(){}
48.   virtual void print(ostream& out) const { out << "A"; }
49.   friend ostream& operator<<(ostream& out, const A& a) {
50.     a.print(out); return out;
51.   }
52. };
53.
54. class B: public A {
55. public:
56.   void print(ostream& out) const override { out << "B"; }
57. };
58.
59. int main() {
60.     MyList<int> m1;
61.     m1.add(9);
62.     m1.add(8);
63.     for(auto val : m1) {
64.         cout << val << endl;
65.     }
66.
67.     MyList<A*> m2;
68.     m2.add(new A());
69.     m2.add(new B());
70.     for(auto val : m2) {
71.         cout << *val << endl;
72.     }
73. }
```

קוד C – לשאלות רשות

```
template<class E> class MyList {
  class Node {
    friend class MyList<E>;
    E data;
    Node* next = nullptr;
    ~Node() { delete next; }
  };
  Node* head = new Node;
public:
  MyList() = default;
  MyList(const MyList& list) {
      list.apply ([this](const auto& val) { add(val); });
  }
  MyList& operator=(MyList list) {
      std::swap(head, list.head);
      return *this;
  }
  MyList(MyList&& list) {
      std::swap(head, list.head);
  }
  void add(const E& data) {
    Node* node = new Node;
    node->data = data;
    node->next = head->next;
    head->next = node;
  }
  ~MyList() { delete head; }
  template<class Function>
  void apply(Function func) const {
    Node* node = head->next;
    while(node != nullptr) {
        func(node->data);
        node = node->next;
    }
  }
};
```

// continues next page

```
int main() {
    MyList<std::string> m3;
    m3.add("Hello");
    m3.add("World");
    MyList<MyList<std::string>> m4;
    m4.add(m3);
    m4.add(std::move(m3));
    m4.apply (
        [](const auto& list){
            list.apply (
                [](const auto& val){
                    cout << val << endl;
                });
        }
    );
}
```

בהצלחה !